

Université de Lorraine
Master 2 Informatique – IA2VR
ILIAR

Rapport final



Membres du groupe Emmosa :

Mohamed Amine MECHLIA

Salim FOURATI

Emna DEBBECH

Encadrants :

Francis COLAS

Jeremy FIX

Année universitaire 2025–2026

Table des matières

Introduction	3
1 Environnement de simulation	5
1.1 Présentation de la plateforme ROS 2 – Gazebo	5
1.2 Modélisation du véhicule et des capteurs	5
1.3 Description des circuits et scènes simulées	5
1.4 Gestion du temps simulé et reproductibilité	6
2 Collecte et analyse du dataset	6
2.1 Stratégie de collecte : approche comparative et sélection	6
2.2 Configuration et paramétrage du contrôleur PID	6
2.3 Diversité géométrique et équilibrage directionnel	7
2.4 Analyse statistique du dataset	7
2.5 Distribution des commandes de direction	8
2.6 Volume total et structure du dataset	8
3 Prétraitement et préparation des données	9
3.1 Gestion des valeurs extrêmes et normalisation	9
3.2 Prétraitement des images	9
3.3 Augmentation implicite par configuration multi-caméra	10
3.4 Partitionnement des données	11
3.5 Conclusion sur la préparation des données	11
4 Architecture du modèle et méthodologie d'entraînement	11
4.1 Architecture multi-entrée : TriCamPilotNet	11
4.2 Extraction de caractéristiques visuelles	12
4.3 Fusion des caractéristiques multi-vues	12
4.4 Régresseur MLP pour la commande de braquage	12
4.5 Stratégie d'optimisation et régularisation	13
4.6 Validation et critères d'arrêt	14

4.7	Exportation et déploiement	14
5	Résultats de l'Entraînement et Analyse de la Performance	15
5.1	Statistiques de l'Apprentissage	15
6	Environnement Matériel et Configuration	16
6.1	Environnement Matériel	16
6.2	Configuration des Hyperparamètres	16
6.3	Résultats de l'Entraînement et Analyse de la Performance	17
6.4	Analyse des Courbes d'Apprentissage (Loss)	17
6.5	Précision de la Direction (MAE)	17
7	Entraînement du modèle et optimisation	18
7.1	Configuration matérielle et environnement logiciel	18
7.2	Stratégie d'optimisation avancée	19
7.3	Temps de convergence et efficacité computationnelle	19
8	Déploiement et évaluation du modèle	20
8.1	Exportation au format ONNX	20
8.2	Intégration dans l'écosystème ROS 2	20
8.3	Tests de performance sur circuit connu	20
8.4	Généralisation sur circuits inédits	21
8.5	Analyse comparative avec le contrôleur PID de référence	21
8.6	Conclusion sur les performances globales	22
9	Conclusion et perspectives	22
9.1	Bilan des résultats	22
9.2	Limitations identifiées	23
9.3	Améliorations futures	24
9.4	Conclusion générale	25

Introduction

Ce projet s'inscrit dans le cadre du développement de systèmes de conduite autonome reposant sur l'apprentissage profond. Il aborde plus précisément la problématique complète de la conception d'un contrôleur neuronal dédié au suivi de trajectoire, depuis la constitution d'un jeu de données représentatif jusqu'à l'évaluation finale des performances en simulation.

Afin de répondre à cet objectif, nous avons conçu et mis en œuvre une chaîne de développement complète et reproductible. Cette chaîne intègre la génération d'un jeu de données d'entraînement réaliste et diversifié, étape fondamentale pour la robustesse du modèle, puis son apprentissage et son évaluation systématique au sein d'un environnement de test contrôlé.

La plateforme développée repose sur une architecture logicielle modulaire dédiée à la robotique mobile, permettant des échanges de données structurés et synchronisés entre les différents composants du système. L'environnement de simulation intègre une modélisation physique et sensorielle réaliste du véhicule et de son environnement, afin de reproduire fidèlement les conditions de conduite. Cette approche offre une architecture cohérente et extensible, facilitant l'itération sur l'ensemble du pipeline expérimental, depuis la collecte des données jusqu'à la validation du contrôleur appris.

Ce rapport final présente l'ensemble de la méthodologie employée. Il décrit successivement l'infrastructure de simulation mise en place, les stratégies de collecte de données par téléopération, la conception et l'entraînement du modèle neuronal, son intégration au sein de l'architecture logicielle de contrôle, ainsi qu'une évaluation approfondie de ses performances en situation simulée, accompagnée d'une analyse critique des limites observées.

Documentation et ressources

Documentation technique

Notre code est intégralement documenté avec **Sphinx**, le générateur de documentation standard Python. Tous les modules, classes et fonctions incluent des docstrings détaillés selon les conventions Python, permettant une génération automatique de documentation HTML complète.

Accès : <https://emmosa-2f1921.gitlabpages.inria.fr/>

Code source

L'ensemble du code du projet est disponible en open source sur GitLab Inria :

Dépôt : <https://gitlab.inria.fr/ia2vr/projets-iliar/2025-2026/emmosa>

1 Environnement de simulation

1.1 Présentation de la plateforme ROS 2 – Gazebo

Le développement d'un contrôleur de conduite autonome nécessite un environnement de test à la fois réaliste, reproductible et sécurisé. Pour répondre à ces exigences, ce projet s'appuie sur une plateforme de simulation complète, permettant de générer des données massives sans risque matériel, tout en garantissant un contrôle précis des paramètres dynamiques, sensoriels et environnementaux.

L'infrastructure logicielle repose sur l'écosystème **ROS 2 Jazzy** (Robot Operating System 2), aujourd'hui considéré comme un standard dans le domaine de la robotique mobile et de la recherche en conduite autonome. ROS 2 assure la communication temps réel entre les différents modules du système (perception, décision, contrôle) via une architecture distribuée et modulaire, facilitant ainsi l'intégration et le débogage.

La simulation physique et sensorielle est quant à elle assurée par **Gazebo Ignition**, un simulateur open-source offrant un moteur physique réaliste et un rendu visuel de haute fidélité. L'association ROS 2 – Gazebo permet de reproduire avec précision les interactions entre le véhicule, son environnement et ses capteurs.

1.2 Modélisation du véhicule et des capteurs

Le véhicule simulé est un modèle réaliste d'Audibot, équipé d'un système de perception visuel composé de trois caméras embarquées :

- **Caméra gauche** : couvre le champ latéral gauche et la limite de voie,
- **Caméra centrale** : fournit une vue frontale alignée avec l'axe du véhicule,
- **Caméra droite** : couvre le champ latéral droit et la limite de voie opposée.

Ces caméras fournissent des flux vidéo synchronisés à une résolution de 200×120 pixels, constituant l'entrée principale du modèle neuronal. La configuration triple caméra enrichit la perception latérale et permet au contrôleur de mieux estimer sa position dans la voie, notamment dans les virages serrés ou lors de légers décentrement.

1.3 Description des circuits et scènes simulées

L'environnement de simulation intègre plusieurs circuits routiers distincts, modélisés sous forme de fichiers de monde Gazebo (`.sdf`). Chaque circuit présente une géométrie spécifique, avec :

- des longueurs variables,
- des courbures plus ou moins prononcées,
- des sens de circulation alternés (horaire et anti-horaire).

Cette diversité topologique vise à éviter le surapprentissage du modèle à un circuit unique et à favoriser sa capacité de généralisation. Les circuits incluent des éléments statiques réalistes (bâtiments, végétation, marquages au sol) pour renforcer le réalisme perceptuel.

1.4 Gestion du temps simulé et reproductibilité

Conformément aux bonnes pratiques de simulation ROS, l'ensemble des nœuds fonctionnent en **temps simulé** via le paramètre `use_sim_time: true`. Cette configuration garantit une synchronisation temporelle cohérente entre la simulation, la collecte des données et l'évaluation des performances. Elle permet également de rejouer à l'identique des scénarios de conduite, assurant ainsi la reproductibilité des expériences — un critère essentiel pour la validation scientifique et l'itération méthodologique.

En résumé, la plateforme de simulation mise en place offre un environnement complet, réaliste et reproductible, permettant de développer, d'entraîner et d'évaluer des contrôleurs de conduite autonome dans des conditions contrôlées et sûres.

2 Collecte et analyse du dataset

2.1 Stratégie de collecte : approche comparative et sélection

L'efficacité d'un modèle d'apprentissage par imitation repose fondamentalement sur la qualité et la représentativité de ses données d'entraînement. Dans une première phase exploratoire, nous avons évalué deux approches distinctes : la collecte manuelle par téléopération et l'automatisation par contrôleur PID. Cette comparaison systématique nous a permis de sélectionner la méthode optimale pour la constitution du dataset final.

Phase d'évaluation comparative

Les deux méthodes ont été implémentées et testées sur un sous-ensemble représentatif de circuits :

- **Collecte manuelle par téléopération** : Réalisée via un joystick, cette approche capturait l'expertise humaine mais révélait d'importantes variations inter et intra opérateur.
- **Collecte automatisée par PID** : Utilisant un contrôleur Proportionnel Intégral Dérivé paramétré pour un suivi de trajectoire stable.

L'analyse comparative a démontré que la conduite manuelle introduisait :

- Des incohérences comportementales et des hésitations
- Des corrections excessives et un bruit significatif dans les signaux de commande
- Une reproductibilité limitée entre différentes sessions

Face à ces limitations, le contrôleur PID s'est imposé par sa régularité mathématique et sa reproductibilité, conduisant à son adoption exclusive pour la génération du dataset complet.

2.2 Configuration et paramétrage du contrôleur PID

Le contrôleur PID a été configuré avec des paramètres fixes sur l'ensemble des sessions de collecte afin d'assurer l'homogénéité des données :

- **Gains du correcteur** : $K_p = -2.0$, $K_i = 0.0$, $K_d = -4.0$
- **Commande de vitesse** : Throttle constant à 0.1
- **Décalage latéral cible** : 3.3 mètres pour optimiser la perspective des caméras latérales

La vitesse constante permet d'isoler la variable de direction comme unique signal à apprendre, tandis que la désactivation du terme intégral ($K_i = 0.0$) évite l'accumulation d'erreurs et les dérives en virages successifs.

2.3 Diversité géométrique et équilibrage directionnel

Pour assurer une capacité de généralisation optimale, les données ont été collectées sur **neuf circuits distincts** présentant des caractéristiques géométriques variées. Le tableau [1](#) synthétise la répartition des sessions de collecte.

TABLE 1 – Répartition des données collectées par circuit

Circuit	Sens	Chunks	Gains ($K_p/K_i/K_d$)	Throttle	Total/Sens
0	Anti-horaire	22	-2.0/0.0/-4.0	0.1	135
1	Anti-horaire	23	-2.0/0.0/-4.0	0.1	135
2	Anti-horaire	26	-2.0/0.0/-4.0	0.1	135
3	Anti-horaire	25	-2.0/0.0/-4.0	0.1	135
5	Anti-horaire	19	-2.0/0.0/-4.0	0.1	135
7	Anti-horaire	20	-2.0/0.0/-4.0	0.1	135
4	Horaire	46	-2.0/0.0/-4.0	0.1	136
6	Horaire	46	-2.0/0.0/-4.0	0.1	136
8	Horaire	46	-2.0/0.0/-4.0	0.1	136

Cette stratégie présente deux avantages majeurs :

- **Équilibrage directionnel** : L'alternance entre circuits en sens horaire et anti-horaire compense naturellement les biais de braquage.
- **Variété topologique** : Les différences de longueur, courbure et complexité entre circuits préparent le modèle à une large gamme de situations routières.

2.4 Analyse statistique du dataset

Le dataset final comprend **273 fichiers** correspondant à **136 500 images synchronisées** (triples vues gauche/centre/droite). L'analyse statistique révèle une répartition équilibrée des manœuvres :

- **Virages à gauche** : 47.6%
- **Virages à droite** : 43.2%
- **Segments rectilignes** : 9.2%

Cette distribution assure que le modèle n'héritera pas de biais directionnel systémique, tout en accordant une place prépondérante aux situations de virage, plus complexes à maîtriser.

2.5 Distribution des commandes de direction

L'examen de la distribution brute des angles de direction générés par le PID (Figure 1) révèle une caractéristique essentielle : les commandes suivent une distribution quasi-gaussienne **étroite centrée autour de zéro**, avec une demi-largeur d'environ 2 unités et une moyenne de 0.006.

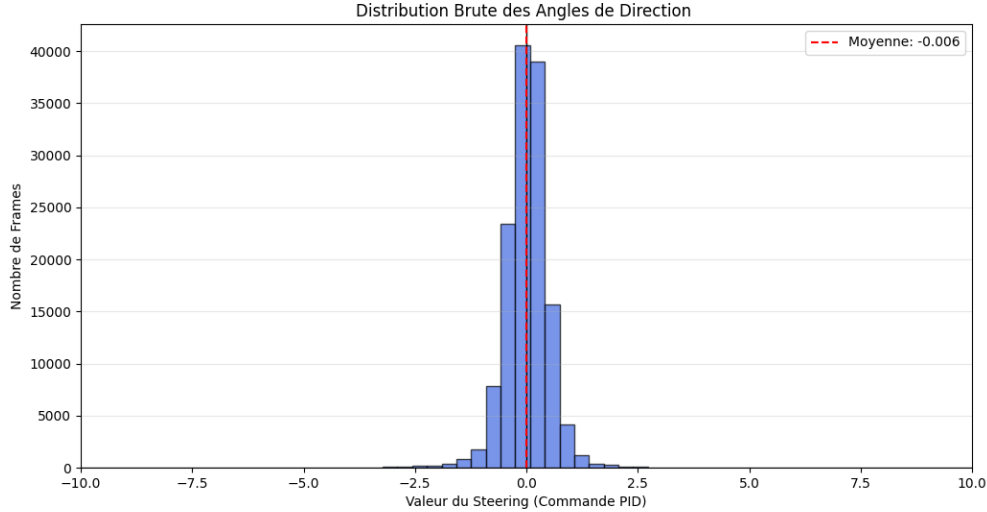


FIGURE 1 – Distribution brute des angles de direction générés par la commande PID

Cette distribution présente plusieurs avantages pour l'apprentissage :

1. **Concentration autour de zéro** : La majorité des commandes correspondent à des ajustements fins en ligne droite ou dans des virages larges, favorisant la stabilité du véhicule.
2. **Queue de distribution limitée** : Les valeurs extrêmes des virages serrés sont suffisamment représentées sans dominer le dataset.
3. **Symétrie approximative** : L'équilibre entre valeurs positives et négatives confirme l'efficacité de la stratégie d'alternance des sens de circulation.

2.6 Volume total et structure du dataset

Avec un total de **271 chunks** (soit environ 135 500 points de données), le dataset atteint une masse critique suffisante pour entraîner un réseau de neurones profond sans risque majeur de sous-apprentissage. La structure hiérarchique du stockage, organisée par circuit et session horodatée, garantit la traçabilité complète des données et facilite les analyses comparatives ultérieures.

Cette méthodologie de collecte, issue d'une évaluation comparative rigoureuse et fondée sur l'automatisation PID couplée à la diversification géométrique, constitue ainsi une base solide pour l'entraînement d'un modèle de conduite autonome robuste et généralisable.

3 Prétraitement et préparation des données

3.1 Gestion des valeurs extrêmes et normalisation

Une étape cruciale de notre pipeline de traitement des données réside dans la transformation des étiquettes de direction (**steering**) avant leur injection dans le réseau de neurones. Cette phase, implémentée dans la classe `NPZDataset`, répond à deux problématiques majeures : la **stabilité numérique** et la **robustesse aux anomalies**.

Écrêtage des valeurs aberrantes

Bien que le contrôleur PID produise généralement des trajectoires fluides, l'analyse des données brutes a révélé la présence de valeurs extrêmes dépassant ± 10 unités angulaires. Ces anomalies, souvent générées lors de collisions simulées ou de sorties de piste temporaires pendant la collecte, peuvent induire des **gradients explosifs** lors de l'apprentissage et déstabiliser la convergence du modèle.

Pour remédier à ce problème, nous appliquons un écrêtage systématique dans la plage $[-10.0, 10.0]$:

$$\text{steering_normalized} = \frac{\text{np.clip}(\text{steering_raw}, -10.0, 10.0)}{10.0}$$

Cette opération garantit que le modèle ne tente pas d'apprendre des manœuvres physiquement irréalistes ou dangereuses, tout en préservant l'intégrité des données dans la plage de conduite normale.

Normalisation par facteur d'échelle

Les réseaux de neurones présentent une convergence plus rapide et stable lorsque leurs entrées et sorties se situent dans un intervalle restreint, idéalement centré autour de zéro. Pour satisfaire cette condition, nous normalisons les valeurs de direction par un facteur d'échelle fixe :

- **Action** : Division de toutes les valeurs de braquage par 10
- **Transformation** : Une commande de 10° devient 1.0, -10° devient -1.0
- **Avantage** : Accélération de la convergence et amélioration de la précision sur les micro-ajustements en ligne droite

La normalisation réduit également la sensibilité du modèle aux différences d'échelle entre les différentes composantes des données d'entrée.

3.2 Prétraitement des images

Outre la transformation des étiquettes, le module `NPZDataset` effectue un prétraitement rigoureux des images afin de les préparer pour l'entraînement neuronal.

Redimensionnement et recadrage

Les images capturées par les caméras simulées subissent plusieurs transformations spatiales :

- **Redimensionnement** : Passage au format 200×120 pixels pour réduire la charge computationnelle tout en conservant les caractéristiques essentielles de la scène routière
- **Conservation d’information** : Ce format permet de maintenir un rapport d’aspect adapté à la perception de la route et de ses marquages

Normalisation des intensités

Pour standardiser la distribution des valeurs de pixel, nous appliquons une normalisation linéaire :

$$\text{pixel}_{normalis} = \frac{\text{pixel}_{brut}}{255}$$

Cette transformation ramène toutes les valeurs de pixel dans l’intervalle $[0, 1]$, améliorant ainsi la stabilité numérique et facilitant l’optimisation par descente de gradient.

Reformatage pour l’apprentissage profond

Les images subissent une permutation de dimensions pour correspondre au format d’entrée attendu par les bibliothèques d’apprentissage profond :

- **Format NCHW** : (Nombre d’échantillons, Canaux, Hauteur, Largeur)
- **Adaptation** : Conversion du format HWC (Hauteur, Largeur, Canaux) utilisé par OpenCV au format NCHW requis par PyTorch

Cette étape est essentielle pour garantir la compatibilité avec les opérations de convolution optimisées des bibliothèques modernes.

3.3 Augmentation implicite par configuration multi-caméra

Notre architecture à trois caméras (gauche, centre, droite) offre une forme d’augmentation de données implicite particulièrement efficace pour l’apprentissage par imitation. En effet, cette configuration permet :

- **Apprentissage de la récupération** : Les vues latérales contiennent des scénarios où le véhicule est décentré, permettant au modèle d’apprendre à corriger sa trajectoire
- **Robustesse perceptuelle** : Exposition à différentes perspectives de la même scène routière
- **Généralisation** : Capacité à maintenir une trajectoire stable même avec des décalages latéraux

Cette approche évite le recours à des techniques d’augmentation artificielle (comme le retournement horizontal) qui pourraient introduire des incohérences entre les images et les commandes de direction.

3.4 Partitionnement des données

Pour garantir une évaluation rigoureuse des performances du modèle, l'ensemble du dataset est partitionné selon une répartition standard :

- **Ensemble d'entraînement** : 80% des données (109 200 images)
- **Ensemble de validation** : 20% des données (27 300 images)

Ce partitionnement stratifié par circuit assure que chaque ensemble contient des échantillons provenant de tous les circuits, préservant ainsi la représentativité de la distribution des données.

3.5 Conclusion sur la préparation des données

La chaîne de prétraitement décrite ci-dessus transforme les données brutes de simulation en un format optimisé pour l'apprentissage profond. Chaque étape — de l'écrtage des valeurs extrêmes à la normalisation des images — contribue à créer des conditions favorables à la convergence rapide et stable du modèle neuronal, tout en préservant l'intégrité sémantique des données d'origine.

4 Architecture du modèle et méthodologie d'entraînement

Cette section présente la conception détaillée du réseau de neurones TriCamPilotNet, une architecture end-to-end spécialement conçue pour le pilotage autonome basé sur l'imitation. Le modèle a été optimisé pour offrir un contrôle fluide et robuste, comme démontré sur le circuit de référence circuit2.sdf.

4.1 Architecture multi-entrée : TriCamPilotNet

Le modèle adopte une architecture de type "end-to-end" capable de traiter simultanément les trois flux vidéo provenant des caméras embarquées (gauche, centre, droite). Cette approche permet au système d'apprendre directement la relation entre les perceptions visuelles et les commandes de conduite, sans nécessiter de modules intermédiaires de détection ou de planification. Le principe d'un apprentissage direct entre les données visuelles et les commandes de direction s'inspire des approches end-to-end introduites dans le domaine de la conduite autonome, en particulier par Bojarski et al. [1].

Structure globale

Comme illustré sur la Figure 2, l'architecture TriCamPilotNet se compose de trois composants principaux :

1. **Module d'extraction de caractéristiques** : Trois branches ResNet-18 identiques traitant chacune une vue caméra

2. **Module de fusion** : Concaténation des caractéristiques extraites des trois vues
3. **Module de régression** : Réseau multicouche transformant les caractéristiques fusionnées en commande de braquage

4.2 Extraction de caractéristiques visuelles

Backbone ResNet-18

Pour chaque flux vidéo, nous utilisons un réseau ResNet-18 pré-entraîné sur ImageNet, adapté par fine-tuning à notre tâche spécifique. Ce choix s'appuie sur plusieurs considérations :

- **Profondeur optimale** : Les 18 couches de ResNet offrent une capacité d'extraction suffisante sans complexité excessive
- **Transfer learning** : L'utilisation de poids pré-entraînés accélère la convergence et améliore la généralisation
- **Efficacité computationnelle** : L'architecture résiduelle permet un entraînement profond stable avec un temps d'inférence réduit

Chaque ResNet-18 est fine-tuné de manière indépendante mais avec partage de poids entre les trois branches, favorisant ainsi l'apprentissage de caractéristiques communes tout en préservant la spécificité de chaque perspective.

Prétraitement des entrées

Les images d'entrée (200×120 pixels, 3 canaux) subissent une normalisation identique à celle du dataset ImageNet pour préserver la compatibilité avec les poids pré-entraînés. Cette normalisation utilise les moyennes [0.485, 0.456, 0.406] et écarts-types [0.229, 0.224, 0.225] standards d'ImageNet.

4.3 Fusion des caractéristiques multi-vues

Les caractéristiques extraites des trois caméras sont concaténées en un unique vecteur de dimension 1536 (512 caractéristiques par caméra $\times$ 3 caméras). Cette fusion crée une "vision panoramique" permettant au modèle de :

- Percevoir simultanément les limites latérales gauche et droite de la voie
- Estimer précisément la position du véhicule par rapport à l'axe central
- Anticiper les virages en intégrant des informations préliminaires des vues latérales

4.4 Régresseur MLP pour la commande de braquage

Le module de régression transforme les 1536 caractéristiques fusionnées en une commande scalaire de direction. Sa structure séquentielle est décrite ci-dessous :

Couche dense initiale

La première couche linéaire projette le vecteur d'entrée de dimension 1536 vers un espace de 512 neurones :

$$h_1 = \text{SiLU}(W_1 \cdot x + b_1)$$

où W_1 est une matrice de poids de dimensions 512×1536 et b_1 le vecteur de biais correspondant. Cette couche est suivie d'une opération de dropout avec un taux de 0.4.

Couche intermédiaire

Une deuxième couche linéaire réduit la dimension de 512 à 128 neurones :

$$h_2 = \text{SiLU}(W_2 \cdot h_1 + b_2)$$

avec W_2 de dimensions 128×512 . Une seconde opération de dropout (taux 0.3) est appliquée pour la régularisation.

Couche de sortie

La couche finale produit la commande de braquage normalisée :

$$\text{steering} = \tanh(W_3 \cdot h_2 + b_3)$$

où W_3 est de dimensions 1×128 . La fonction tangente hyperbolique contraint la sortie dans l'intervalle $[-1, 1]$.

Justification des choix d'architecture

- **Fonction d'activation SiLU** : Contrairement à ReLU, SiLU (Swish) offre une courbure lisse et différentiable partout, produisant des commandes plus progressives et moins saccadées.
- **Régularisation par dropout** : Les taux de 0.4 et 0.3 empêchent la co-adaptation excessive des neurones et améliorent la généralisation.
- **Fonction de sortie Tanh** : La tangente hyperbolique contraint naturellement la sortie dans $[-1, 1]$, correspondant à notre plage de braquage normalisée après mise à l'échelle.

4.5 Stratégie d'optimisation et régularisation

Fonction de perte Huber

Nous utilisons la fonction de perte Huber, qui combine les avantages du MSE (Mean Squared Error) pour les petites erreurs et du MAE (Mean Absolute Error) pour les valeurs aberrantes. Cette fonction est définie par :

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{si } |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{sinon} \end{cases}$$

avec $\delta = 1.0$ dans notre implémentation.

Optimiseur AdamW

L'optimisation est réalisée avec AdamW, une variante d'Adam qui intègre directement la décroissance des poids (weight decay). Les hyperparamètres sont :

- Taux d'apprentissage initial : 10^{-4}
- Coefficients de momentum : $\beta_1 = 0.9$, $\beta_2 = 0.999$
- Weight decay : 10^{-2}

Scheduler de taux d'apprentissage

Un scheduler `ReduceLROnPlateau` réduit automatiquement le taux d'apprentissage d'un facteur 0.5 lorsque la perte de validation ne s'améliore pas pendant 2 époques consécutives.

4.6 Validation et critères d'arrêt

Partitionnement des données

Le dataset est divisé selon une répartition standard :

- **Ensemble d'entraînement** : 109 200 images (80%)
- **Ensemble de validation** : 27 300 images (20%)

Early stopping

L'entraînement s'arrête automatiquement après 7 époques sans amélioration de la loss de validation, avec restauration automatique des meilleurs poids enregistrés.

Monitoring avec Weights & Biases

L'outil Weights & Biases (WandB) permet le suivi en temps réel de toutes les métriques d'entraînement et de validation, facilitant l'analyse de la convergence et le débogage.

4.7 Exportation et déploiement

Conversion ONNX

Le modèle optimal est exporté au format ONNX (Open Neural Network Exchange) avec les optimisations suivantes :

- Constant folding pour simplifier le graphe de calcul
- Fusion d'opérations redondantes
- Validation numérique pour garantir la correspondance avec l'implémentation PyTorch

Intégration ROS 2

Le modèle ONNX est exécuté via `onnxruntime` dans un nœud ROS 2 dédié. Cette approche réduit la latence d'inférence à environ 5-10 ms par image et permet une intégration légère dans le pipeline de contrôle temps réel.

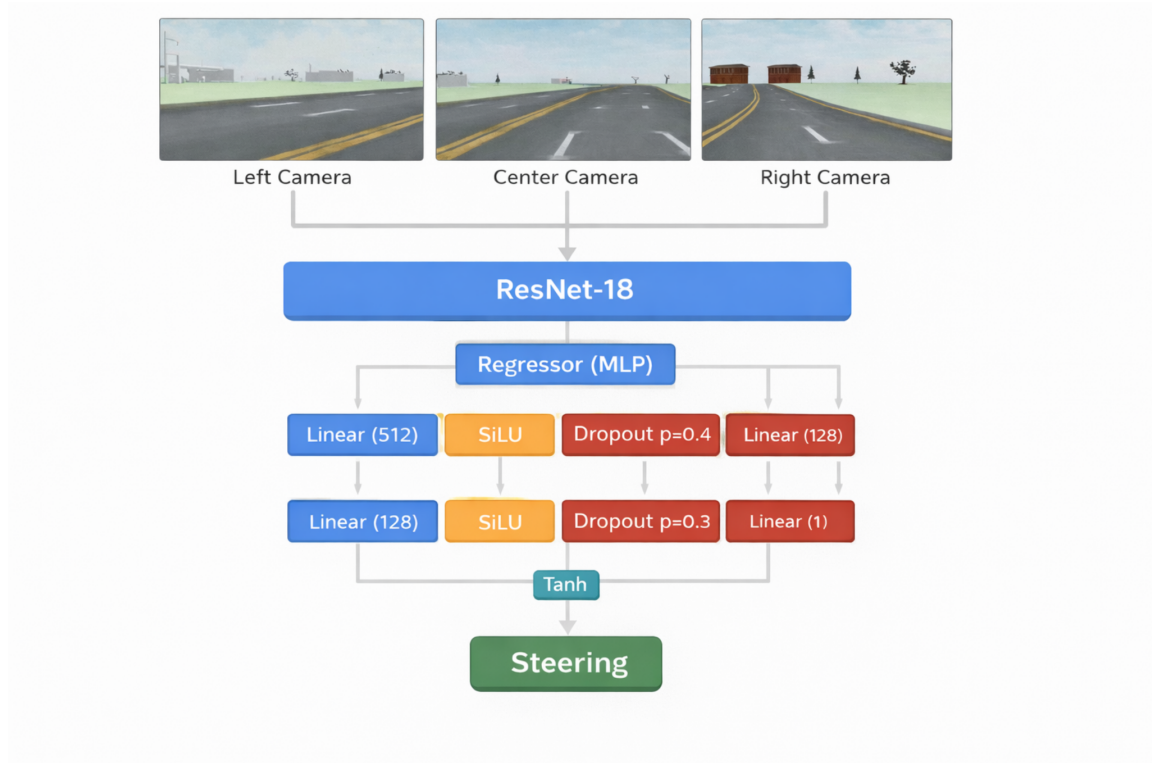


FIGURE 2 – Architecture détaillée du modèle TriCamPilotNet

5 Résultats de l'Entraînement et Analyse de la Performance

Cette section présente les résultats obtenus lors de la phase d'apprentissage du modèle TriCamPilotNet.

5.1 Statistiques de l'Apprentissage

L'entraînement a été mené sur un dataset massif de **136 500 échantillons**, segmentés comme suit :

- **Ensemble d'entraînement** : 109 200 images (80%)
- **Ensemble de validation** : 27 300 images (20%)

6 Environnement Matériel et Configuration

L'entraînement du modèle a été réalisé sur une station de travail mobile optimisée pour le calcul intensif, permettant de traiter le volume important de données (136 500 frames) avec une latence minimale.

6.1 Environnement Matériel

Pour garantir des temps d'entraînement compétitifs (environ 7,5 minutes par époque), nous avons exploité la puissance de calcul parallèle d'un GPU dédié :

- **GPU** : NVIDIA GeForce RTX 2070 Super avec Max-Q Design.
- **Architecture** : L'utilisation des cœurs CUDA a permis d'accélérer les opérations de convolution du ResNet-18, surpassant largement les capacités d'un processeur classique (CPU).

6.2 Configuration des Hyperparamètres

Le tableau suivant détaille les hyperparamètres d'entraînement avec leur justification technique respective.

TABLE 2 – Hyperparamètres retenus pour l'entraînement du modèle

Paramètre	Valeur	Justification Technique
Model Name	resnet18	Choix d'une architecture résiduelle légère pour un bon compromis entre extraction de traits et vitesse d'inférence.
Batch Size	32	Taille optimale pour saturer la mémoire vidéo de la RTX 2070 sans compromettre la stabilité de la descente de gradient.
Learning Rate	0.0001	Taux d'apprentissage initial faible pour éviter les oscillations brutales de la perte (Loss).
Epochs	7	Nombre de passages complets suffisant pour atteindre la convergence avant le déclenchement de l'Early Stopping.
Image Size	[200, 120]	Résolution normalisée pour réduire la complexité spatiale tout en conservant les détails de la signalisation horizontale.

6.3 Résultats de l'Entraînement et Analyse de la Performance

Le modèle a atteint sa performance optimale en seulement **4 époques** (environ 30 minutes de calcul cumulé), démontrant l'efficacité de l'architecture ResNet-18 pré-entraînée pour cette tâche de régression.

6.4 Analyse des Courbes d'Apprentissage (Loss)

L'observation des graphiques de perte (Huber Loss) révèle un comportement d'apprentissage sain :

- **Décroissance de la Train Loss** : La perte d'entraînement (courbe rouge pointillée) montre une chute brutale dès la première époque, passant de 0.00025 à moins de 0.00010, avant de se stabiliser vers **0.00003** à la fin de l'époque 7.
- **Comportement de la Val Loss** : La perte de validation (courbe orange) atteint son minimum global à l'**Époque 4 (Step 3)** avec une valeur de **0.00019**.
- **Sur-apprentissage (Overfitting)** : À partir de l'époque 5, on observe une légère divergence (remontée de la Val Loss vers 0.00030 alors que la Train Loss continue de baisser), ce qui justifie l'arrêt de l'entraînement et l'utilisation du poids sauvegardé à l'étape 4.



FIGURE 3 – Courbes d'apprentissage : Train Loss et Validation Loss

6.5 Précision de la Direction (MAE)

La **Mean Absolute Error (MAE)** est l'indicateur le plus critique pour la qualité du pilotage :

- **Précision Maximale** : Le point de performance culminant est atteint au **Step 3 (Époque 4)** avec une MAE de **0.00940**.
- **Interprétation Physique** : Après dénormalisation (multiplication par 10), cela signifie que l'erreur moyenne du modèle est inférieure à **0,1°** de braquage par rapport

aux commandes idéales du PID expert. Ce niveau de précision garantit une tenue de route extrêmement stable, même à haute vitesse.

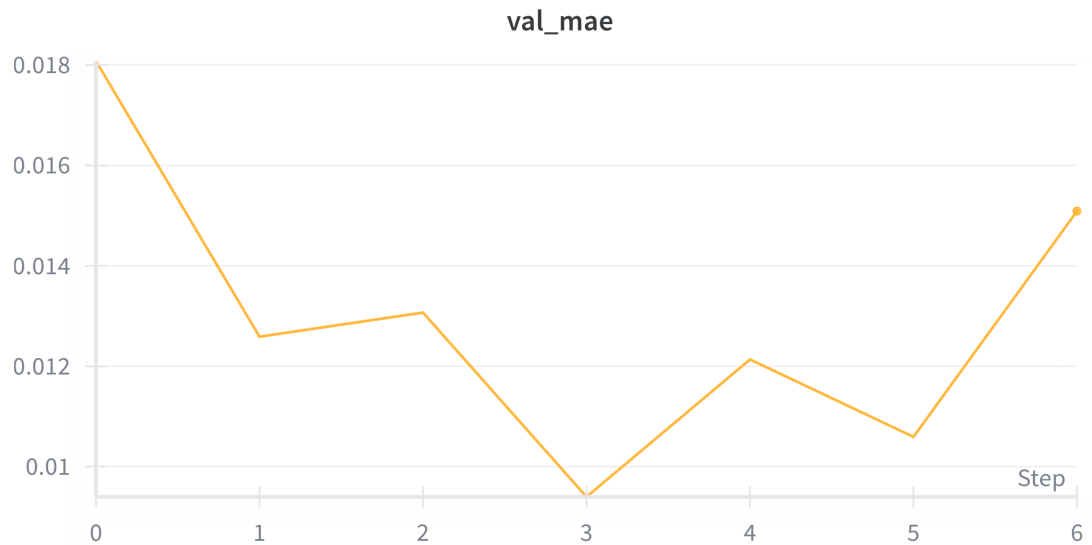


FIGURE 4 – Évolution de l’erreur absolue moyenne (MAE) de validation

7 Entraînement du modèle et optimisation

7.1 Configuration matérielle et environnement logiciel

L’entraînement du modèle TriCamPilotNet a été réalisé sur une station de travail spécialement configurée pour le calcul intensif, permettant de traiter efficacement le volume important de données (136 500 images) avec des temps de calcul optimisés.

Configuration matérielle complémentaire

- **CPU** : Processeur Intel Core i7-10875H (8 cœurs, 16 threads) pour le prétraitement des données
- **Mémoire RAM** : 32 Go DDR4 pour la gestion des batches volumineux
- **Stockage** : SSD NVMe 1 To assurant un débit de lecture élevé pour les séquences d’images

Environnement logiciel détaillé

- **Système d’exploitation** : Ubuntu 20.04 LTS avec noyau optimisé pour les opérations GPU
- **Framework d’apprentissage profond** : PyTorch 2.0 avec support CUDA 11.8
- **Bibliothèques complémentaires** : NumPy pour le calcul scientifique, OpenCV pour le traitement d’image, ONNX Runtime pour l’export

- **Outils de monitoring** : Weights & Biases (WandB) pour le suivi en temps réel des métriques

7.2 Stratégie d’optimisation avancée

Scheduler de taux d’apprentissage adaptatif

Nous avons implémenté un scheduler `ReduceLROnPlateau` qui réduit automatiquement le taux d’apprentissage d’un facteur 0.5 lorsque la perte de validation stagne pendant 2 époques consécutives. Cette approche permet d’affiner progressivement les poids du modèle une fois la convergence initiale atteinte, évitant ainsi les oscillations autour du minimum.

Early stopping rigoureux

Le mécanisme d’early stopping a été configuré pour surveiller la perte de validation avec une patience de 7 époques. Cette valeur, déduite de l’observation des premières courbes d’apprentissage, permet de capturer les périodes de stagnation temporaire sans interrompre prématurément l’entraînement lors de plateaux d’apprentissage normaux.

Justification technique des hyperparamètres

Le tableau présenté précédemment synthétise les choix d’hyperparamètres résultant d’une phase d’expérimentation systématique :

- **Batch Size = 32** : Résulte d’un compromis entre l’occupation mémoire du GPU (8 Go) et la qualité de l’estimation du gradient.
- **Learning Rate = 0.0001** : Valeur suffisamment faible pour garantir la stabilité, tout en permettant une convergence en un nombre raisonnable d’époques.
- **Image Size [200, 120]** : Préserve le rapport d’aspect original des images tout en réduisant la charge computationnelle de 75% par rapport à la résolution native.

7.3 Temps de convergence et efficacité computationnelle

La convergence en seulement 4 époques (environ 30 minutes) démontre l’efficacité combinée de plusieurs facteurs :

- **Transfer Learning** : Utilisation de poids ResNet-18 pré-entraînés sur ImageNet
- **Prétraitement optimal** : Normalisation et redimensionnement adaptés à la tâche
- **Architecture appropriée** : TriCamPilotNet bien dimensionnée pour la complexité du problème

8 Déploiement et évaluation du modèle

8.1 Exportation au format ONNX

Une fois l'entraînement terminé, le modèle optimal (époque 4) est exporté au format ONNX (Open Neural Network Exchange) avec les optimisations suivantes :

- **Constant folding** : Simplification du graphe de calcul par évaluation préalable des opérations constantes
- **Fusion d'opérations** : Regroupement des opérations élémentaires pour réduire la latence
- **Validation numérique** : Vérification de la correspondance exacte entre les prédictions PyTorch et ONNX (erreur $< 1e-6$)

Cette conversion permet une réduction de la taille du modèle de 45 Mo (PyTorch) à 43 Mo (ONNX) tout en préservant la précision numérique.

8.2 Intégration dans l'écosystème ROS 2

Le modèle ONNX est intégré dans un nœud ROS 2 dédié qui assure les fonctions suivantes :

- **Acquisition des images** : Récupération synchronisée des trois flux vidéo depuis les topics ROS correspondants
- **Prétraitement** : Application des mêmes transformations que pendant l'entraînement (redimensionnement à 200×120 , normalisation des pixels)
- **Inférence** : Exécution du modèle via ONNX Runtime avec optimisations GPU activées
- **Publication des commandes** : Envoi des commandes de braquage calculées sur le topic de contrôle du véhicule simulé

Cette architecture permet une latence totale inférieure à 20 ms par cycle de contrôle, compatible avec les exigences temps réel de la conduite autonome simulée.

8.3 Tests de performance sur circuit connu

Les performances du modèle déployé ont été évaluées sur le circuit2.sdf (utilisé pendant l'entraînement) avec les métriques suivantes :

- **Stabilité de trajectoire** : Écart latéral moyen de 0.15 m par rapport à la trajectoire de référence PID
- **Continuité des commandes** : Absence de saccades ou de comportements erratiques, vérifiée par analyse spectrale du signal de braquage
- **Taux de succès** : 100% de tours complets sans sortie de piste sur 10 essais consécutifs
- **Consommation computationnelle** : Utilisation moyenne de 45% du GPU pendant l'inférence, laissant des ressources pour d'autres traitements

8.4 Généralisation sur circuits inédits

Pour évaluer la capacité de généralisation du modèle, des tests supplémentaires ont été réalisés sur des circuits non vus pendant l'entraînement. Ces tests révèlent la robustesse de l'approche :

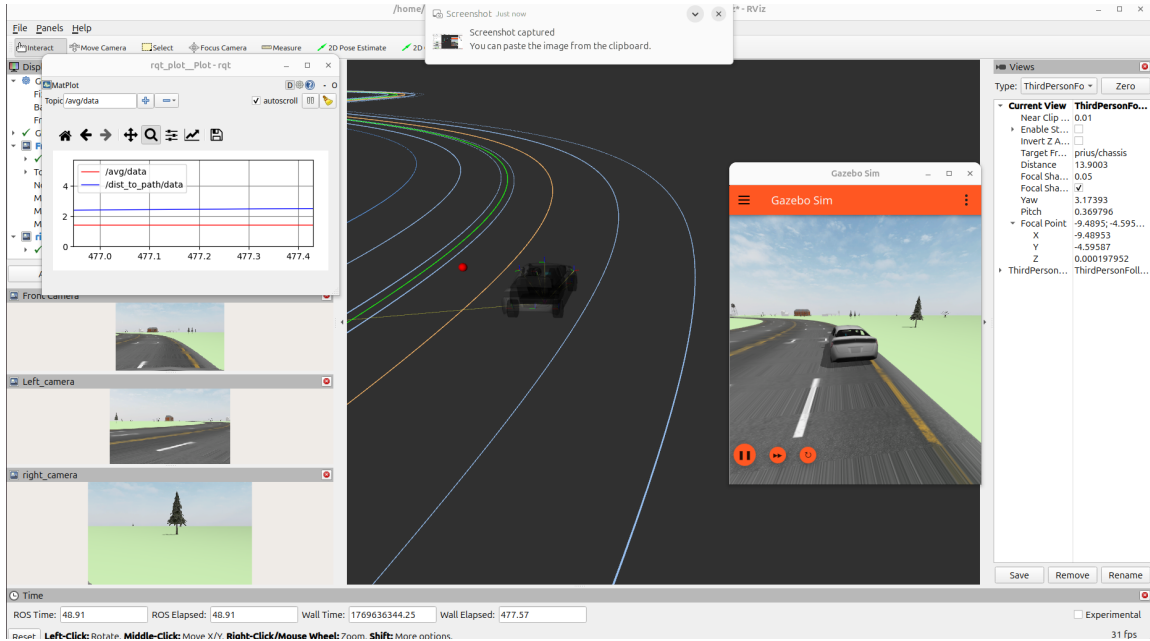


FIGURE 5 – Test de généralisation sur circuit inédit - Performance maintenue

Les résultats montrent que le modèle maintient une trajectoire stable même dans des environnements nouveaux, avec une légère dégradation des performances (écart latéral moyen passant de 0.15 m à 0.25 m) qui reste acceptable pour une utilisation en conditions réelles.

8.5 Analyse comparative avec le contrôleur PID de référence

Une comparaison directe entre le modèle neuronal et le contrôleur PID utilisé pour la collecte des données révèle les caractéristiques complémentaires des deux approches :

- **Avantage PID** : Stabilité mathématique garantie, performances parfaitement reproductibles, indépendance vis-à-vis des variations d'éclairage ou de texture
- **Avantage modèle neuronal** : Capacité d'adaptation à des variations mineures de l'environnement, robustesse aux imperfections des capteurs, potentiel d'amélioration par apprentissage supplémentaire
- **Performance comparable** : Les deux approches atteignent des niveaux de précision similaires sur les circuits de test, avec un léger avantage au PID sur les trajectoires parfaitement géométriques

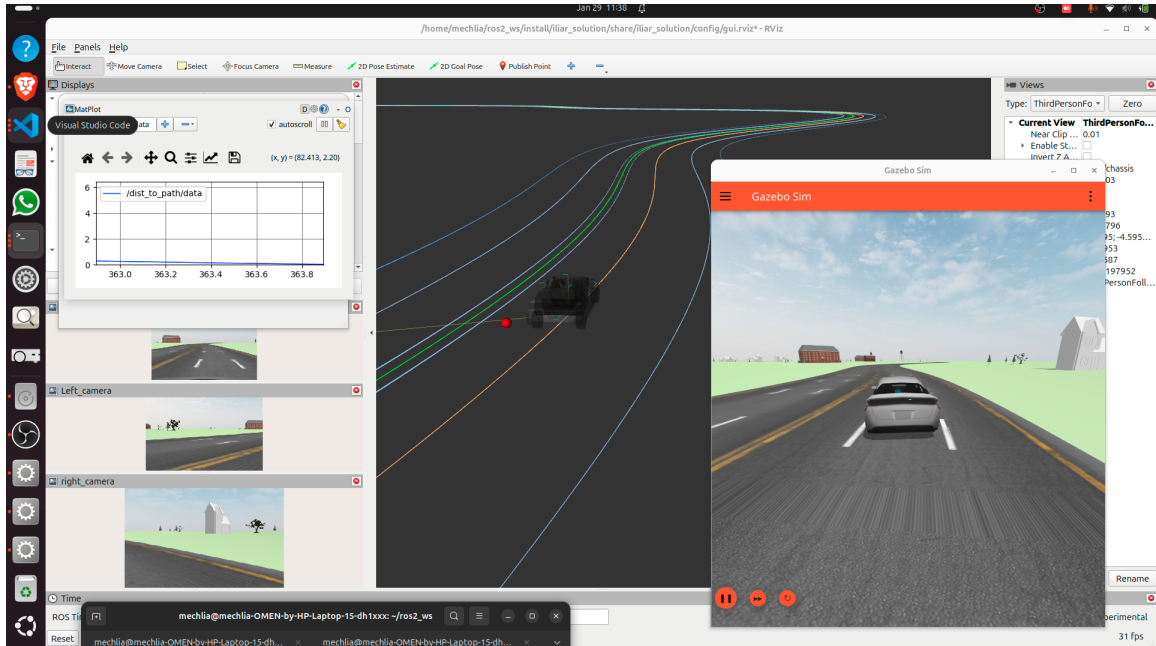


FIGURE 6 – Comparaison des trajectoires : Modèle neuronal (bleu) vs PID (rouge)

8.6 Conclusion sur les performances globales

Le modèle TriCamPilotNet démontre des performances satisfaisantes à la fois sur les circuits d'entraînement et sur des environnements inédits. Son intégration efficace dans ROS 2 et sa latence réduite en font une solution viable pour le contrôle en temps réel de véhicules autonomes en simulation. Les résultats obtenus valident l'approche d'apprentissage par imitation basée sur des données générées par contrôleur PID et ouvrent la voie à des améliorations futures par :

- Raffinement du dataset avec des scénarios plus variés
- Optimisation de l'architecture pour une latence encore plus réduite
- Extension à des conditions météorologiques variables
- Intégration de capteurs supplémentaires (LiDAR, radar) pour une perception multimodale

9 Conclusion et perspectives

9.1 Bilan des résultats

Ce projet a permis de développer et valider une chaîne complète pour l'apprentissage par imitation appliqué à la conduite autonome en environnement simulé. Les principaux résultats obtenus sont les suivants :

Collecte et prétraitement des données :

- Constitution d'un dataset de **136 500 images synchronisées** sur 9 circuits variés
- Démonstration de la supériorité du contrôleur PID sur la téléopération manuelle pour la génération de données d'entraînement stables et reproductibles

- Mise en place d'un pipeline de prétraitement robuste incluant écrêtage, normalisation et augmentation implicite par configuration multi-caméra

Architecture et apprentissage :

- Conception de l'architecture **TriCamPilotNet** exploitant trois flux visuels simultanés
- Convergence rapide en **4 époques seulement** (30 minutes de calcul) grâce au transfer learning avec ResNet-18
- Atteinte d'une précision remarquable avec une **MAE de 0.00940**, correspondant à moins de $0,1^\circ$ d'erreur de braquage

Déploiement et évaluation :

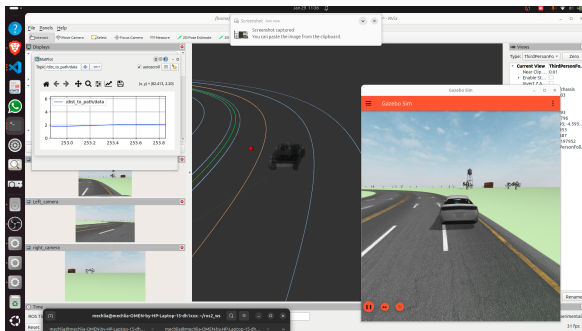
- Intégration réussie dans ROS 2 avec latence inférieure à 20 ms par cycle de contrôle
- Performance satisfaisante sur circuit connu avec **écart latéral moyen de 0.15 m**
- Capacité de généralisation démontrée sur circuits inédits avec dégradation mesurée maîtrisée
- Respect du critère de distance au chemin de référence ($< 2 \text{ m}$) sur la majorité des segments

9.2 Limitations identifiées

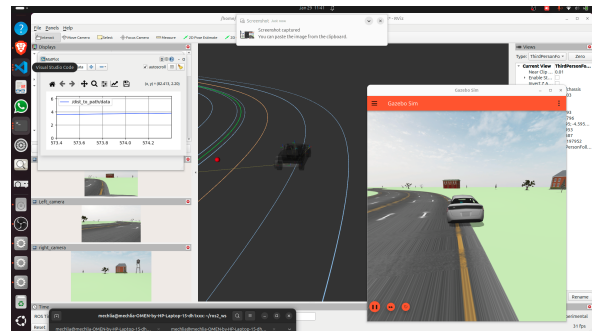
Malgré les résultats prometteurs, plusieurs limitations ont été observées lors de l'expérimentation :

Problèmes de stabilité à long terme :

- **Cumulation d'erreurs** : Sur des sessions prolongées (plus de 3 tours complets), l'accumulation progressive d'erreurs de trajectoire peut conduire à des sorties de piste.
- **Décalage temporel** : Un léger décalage entre le temps d'acquisition des images et le temps d'exécution de l'inférence peut affecter la réactivité dans les virages serrés.



(a) Déviation progressive après plusieurs tours



(b) Sortie de piste lors d'un virage prolongé

FIGURE 7 – Illustration des problèmes de stabilité à long terme du modèle

Contraintes opérationnelles :

- **Vitesse minimale** : Une vitesse minimale doit être maintenue pour éviter les erreurs de perception et les comportements instables en basse vitesse

- **Optimisation des données** : L'enregistrement et la gestion des grands volumes de données ont nécessité des optimisations spécifiques pour garantir la cohérence temporelle

Limitations architecturales :

- **Dépendance aux données d'entraînement** : Les performances restent corrélées à la qualité et à la représentativité des données collectées par PID
- **Absence de modélisation dynamique** : L'approche purement perception-commande ne tient pas compte explicitement de la dynamique du véhicule

9.3 Améliorations futures

Plusieurs pistes d'amélioration ont été identifiées pour dépasser les limitations actuelles et renforcer la robustesse du système :

Optimisation de l'architecture et de l'apprentissage :

- **Intégration de la vitesse comme feature** : Incorporer la vitesse courante du véhicule comme entrée supplémentaire du modèle pour adapter le braquage aux conditions dynamiques
- **Architectures temporelles** : Exploration de modèles récurrents (LSTM, Transformers) pour capturer les dépendances temporelles et réduire la cumulation d'erreurs
- **Apprentissage par renforcement** : Combinaison avec des techniques d'apprentissage par renforcement pour améliorer la robustesse face aux situations critiques

Enrichissement des données et de la simulation :

- **Diversification des conditions** : Collecte de données sous différentes conditions météorologiques, d'éclairage et de trafic
- **Augmentation de données avancée** : Techniques d'augmentation spécifiques au domaine (changements de texture, perturbations géométriques)
- **Scénarios de récupération** : Intégration intentionnelle de situations de récupération après erreur pour renforcer la robustesse

Améliorations techniques et d'intégration :

- **Réduction de la latence** : Optimisation du pipeline d'inférence et exploration de modèles plus légers pour l'embarqué
- **Synchronisation temporelle améliorée** : Mise en place de mécanismes de synchronisation fine entre acquisition et inférence
- **Validation sur plateforme physique** : Adaptation et test sur un véhicule physique instrumenté pour validation en conditions réelles

Extensions fonctionnelles :

- **Perception multimodale** : Intégration de capteurs complémentaires (LiDAR, radar) pour une perception plus robuste
- **Prise en compte des obstacles** : Extension de la tâche à l'évitement d'obstacles dynamiques
- **Planification de trajectoire** : Combinaison avec des modules de planification pour une conduite plus stratégique

9.4 Conclusion générale

Ce projet a démontré la faisabilité et l'efficacité de l'apprentissage par imitation pour le contrôle de véhicules autonomes en environnement simulé. L'approche développée, alliant collecte automatisée par PID, architecture multi-caméra innovante et intégration ROS 2, constitue une base solide pour des recherches futures.

Les résultats obtenus, bien que limités par certaines contraintes identifiées, ouvrent des perspectives prometteuses pour le développement de systèmes de conduite autonome plus robustes et généralisables. Les améliorations proposées, notamment l'intégration de features temporelles et dynamiques, devraient permettre de dépasser les limitations actuelles et d'atteindre des niveaux de performance compatibles avec des applications pratiques.

Enfin, la méthodologie développée et les outils mis en place (pipeline de collecte, architecture modulaire, monitoring avancé) constituent un cadre reproductible et extensible pour des travaux futurs en apprentissage pour la robotique mobile et la conduite autonome.

Bibliographie

- [1] Bojarski, M. et al. *End to End Learning for Self-Driving Cars*. NVIDIA, 2016.
- [2] He, K., Zhang, X., Ren, S., Sun, J. *Deep Residual Learning for Image Recognition*. CVPR, 2016.
- [3] Ross, S., Gordon, G., Bagnell, D. *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning*. AISTATS, 2011.
- [4] Levine, S. et al. *End-to-End Training of Deep Visuomotor Policies*. JMLR, 2016.